

Software Requirements Specification

OptiEDT

AI-assisted generation, ranking and explanation of university timetables

Author	Mohamed Jawad Tourir		
Academic year	2025 – 2026	Duration	4 weeks
Project	Company internship project	Date	26 July 2026
Field of application	Tunisian public universities (LMD system)		

Contents

1. Introduction.....	4
1.1 Purpose of the document.....	4
1.2 Scope of the system	4
1.3 Definitions and abbreviations.....	4
1.4 References.....	5
2. General description.....	5
2.1 Overview of the system	5
2.2 Users of the system.....	6
2.3 Operating environment	7
2.4 Assumptions and dependencies.....	7
3. Functional requirements	8
3.1 Summary of the requirements.....	8
3.2 Detailed specification	9
3.3 Main flow	12
4. External interfaces	13
4.1 User interfaces.....	13
4.2 Software interfaces.....	14
4.3 Communication interfaces.....	14
5. Data model	14
5.1 Main entities.....	14
5.2 Rules of the model.....	16
6. Constraint model.....	17
6.1 Decision variables	17
6.2 Hard constraints	17
6.3 Order of the constraints and the calendar	18
6.4 Objective function	18
6.5 Size of the model.....	18
6.6 Weight profiles and portfolio	19

6.7 Score, order and decomposition of the difference	19
6.8 Model of the examination session	20
6.9 Adjustment of the weights.....	20
6.10 Recommendation and regeneration model	21
6.11 AI assistant service	22
6.11.1 Construction of the context.....	22
6.11.2 Verification of the answer	22
6.11.3 Suggestions produced by the assistant	23
7. Non-functional requirements.....	23
7.1 Performance	23
7.2 Reliability.....	24
7.3 Security.....	24
7.4 Maintainability and portability.....	24
7.5 Requirements applicable to the analysis and learning components	24
8. Validation and tests.....	25
8.1 Validation of the engine.....	25
8.2 Verification of the data.....	25
8.3 Diagnosis of an infeasible instance	26
8.4 Validation of the ranking and of the explanation.....	26
8.5 Evaluation of the adjustment of the weights	26
8.6 Acceptance tests	27
9. Traceability.....	27
10. Annexes.....	28

List of figures

Figure 1. Position of the decision layer and of the analysis layer in the system.	6
Figure 2. Actors and main use cases.	7
Figure 3. Sequence of one generation run, from launch to publication.	13
Figure 4. Entity-relationship diagram of the database.	16
Figure 5. AI recommendation and regeneration loop.	22
Figure 6. AI assistant service, grounded on computed facts	23

List of tables

Table 1. Definitions and abbreviations.....	4
Table 2. Users of the system and their rights.....	6
Table 3. Summary of the functional requirements.....	8
Table 4. FR-1. Data management.....	9
Table 5. FR-2. Collection of availability.....	9
Table 6. FR-3. Generation of a valid timetable.....	9
Table 7. FR-4. Improvement of the criteria.....	9
Table 8. FR-5. Scoring of the candidates.....	9
Table 9. FR-7. Display of the views.....	9
Table 10. FR-8. Naming of the rules in conflict.....	10
Table 11. FR-9. Configuration of the calendar.....	10
Table 12. FR-11. Authentication and rights.....	10
Table 13. FR-12. Verification of the data before solving.....	10
Table 14. FR-13. Production of the candidates under distinct profiles.....	10
Table 15. FR-14. Comparison of two candidates.....	10
Table 16. FR-15. Decomposition of the difference.....	10
Table 17. FR-16. Recommendation.....	11
Table 18. FR-19. Record of the runs.....	11
Table 19. FR-20. Examination session.....	11
Table 20. FR-21. Adjustment of the weights.....	11
Table 21. FR-22. Explanation of a candidate.....	11
Table 22. FR-23. Regeneration from a recommendation.....	11
Table 23. FR-24. Question in ordinary language.....	12
Table 24. FR-25. Report on a run.....	12
Table 25. Main entities of the data model.....	14
Table 26. Decision variables of the model.....	17
Table 27. Hard constraints and their construction.....	17
Table 28. Criteria of quality and default weights.....	18
Table 29. Size of the model on the reference instance.....	18
Table 30. Constraints of the examination model.....	20
Table 31. Catalogue of the recommendation action types.....	21
Table 32. Context supplied to the language model.....	22
Table 33. Result of the five verifications of the data.....	25
Table 34. Properties tested on the analysis layer.....	26
Table 35. Acceptance tests.....	27
Table 36. Traceability of the requirements.....	27

1. Introduction

1.1 Purpose of the document

This document specifies the requirements of the OptiEDT application in a form usable for implementation and testing. It completes the Cahier des Charges : where that document states what the application must do, the present document states how each requirement is expressed in terms of data, interfaces and constraints.

1.2 Scope of the system

- Generation of the weekly timetable of a faculty for the sessions of type CM, TD and TP.
- Collection of the availability declared by the teachers.
- Verification of the data before solving and report of the structural risks found.
- Production of several candidate timetables under distinct weight profiles.
- Scoring, ordering and comparison of the candidates, with the decomposition of the difference between two of them.
- Naming of the rules in conflict when no timetable exists.
- Formulation of AI recommendations attached to a candidate, and regeneration of an improved candidate on acceptance, through the same solving engine.
- AI assistant service : explanation of a candidate, answers to questions asked in ordinary language, and generation of a readable report, all grounded on figures already computed.
- Display of the published timetable by teacher, by group, by classroom and by laboratory.
- Generation of the timetable of an examination session, adjustment of the weights and formulation of an explanation in ordinary language, which constitute the second increment.

1.3 Definitions and abbreviations

Table 1. Definitions and abbreviations

Term	Definition
CM, TD, TP	Lecture for a whole promotion, tutorial for a group, laboratory session for a subgroup
LMD	Licence, Master, Doctorat degree structure
CP-SAT	Constraint satisfaction and optimisation solver of the OR-Tools library
Session	Teaching unit to be placed, characterised by a course, a group, a teacher and a duration
Slot	Time unit of the weekly grid, identified by a single index
Open slot	Slot whose field <code>is_open</code> equals one, that is a slot usable for teaching
Run	One execution of the generation, with its data, its seed, its weights and its results
Weight profile	Vector of non-negative weights over the soft constraints, of sum equal to one
Candidate	Valid timetable produced by a run under one weight profile
Portfolio	Set of the candidates produced by one run

Term	Definition
Sub-score	Value of one criterion of quality for one candidate, brought back to the interval from zero to one
Contribution	Part of the difference between two scores attributable to one criterion
Dominated candidate	Candidate which another candidate improves on every criterion
Comparison	Choice recorded when the person in charge retains one candidate rather than another
Assumption	Literal attached to a constraint, which the solver may return to explain an infeasibility
Hard constraint	Rule which must be respected in every accepted timetable
Soft constraint	Quality criterion whose violation adds a penalty to the cost

1.4 References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
- ISO/IEC 25010, Systems and software Quality Requirements and Evaluation, quality model.
- Di Gaspero, McCollum and Schaerf, The Second International Timetabling Competition, Track 3, Curriculum-based Course Timetabling, technical report, 2007.
- McCollum et al., The Second International Timetabling Competition, Track 1, Examination Timetabling, 2007.
- Bettinelli et al., An overview of curriculum-based course timetabling, TOP, 2015.
- Post, Kingston, Ahmadi et al., XHSTT, an XML archive for high school timetabling problems in different countries, Annals of Operations Research, volume 218, 2014.
- Carter, Laporte and Lee, Examination timetabling, algorithmic strategies and applications, 1996.
- Russell and Norvig, Artificial Intelligence, a Modern Approach, chapter on constraint satisfaction problems.
- Google OR-Tools, CP-SAT documentation, sections on scheduling constraints and on troubleshooting with assumptions.
- Kadzinski et al., On learning of weights through preferences, learning to rank applied to multi-criteria decision making.

2. General description

2.1 Overview of the system

The application is organised in four layers, deployed as a single unit.

- **Presentation layer** : web interface displaying the grids, the comparison of the candidates and the timetable views.
- **Application layer** : server exposing a REST interface, responsible for the data, for the runs and for the calls to the engine.
- **Decision layer** : CP-SAT model built from the data of the database, which alone assigns a slot and a room to a session.

- **Analysis layer** : components which read the candidates produced and return the sub-scores, the order, the decomposition of the differences and, in the second increment, the adjusted weights and the formulated sentence.

The separation between the third and the fourth layer is a requirement and not a description, illustrated in *Figure 1* : no component of the analysis layer may add, move or remove a session. The consequence is that an error in the analysis layer produces a wrong order and never an invalid timetable.

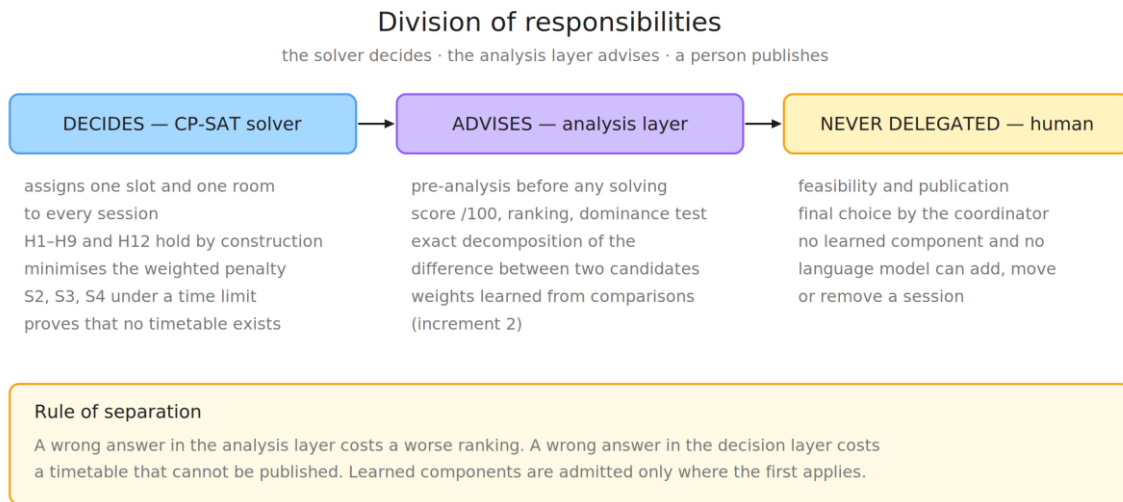


Figure 1. Position of the decision layer and of the analysis layer in the system.

2.2 Users of the system

Table 2. Users of the system and their rights

User	Frequency of use	Rights
Person in charge of the timetable	Intensive at the start of the semester	Read and write on all the data, launch of the runs, comparison and publication
Teacher	Occasional	Read and write on own availability, read own timetable
Student	Occasional	Read on the timetable of the group
Administrator	Occasional	Management of the accounts and of the calendar

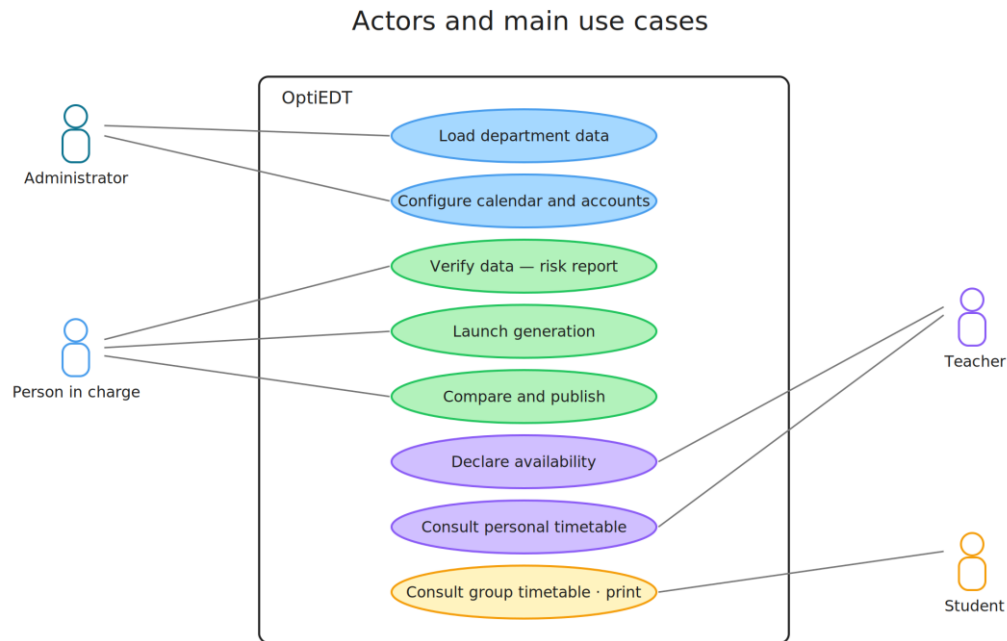


Figure 2. Actors and main use cases.

2.3 Operating environment

- **Server** : standard processor, no graphics card dedicated to computation.
- **Client** : recent web browser, no installation required.
- **Database** : PostgreSQL relational server.
- **Language** : Python for the server, the engine and the analysis components, JavaScript for the interface.
- **Execution of the runs** : background task in the same process as the server, the interface following the state of the run.
- **Optional external service** : interface towards a language model, used only in the second increment and deactivable by configuration.

2.4 Assumptions and dependencies

- The composition of the groups is fixed for the whole semester.
- The list of rooms and the enrolments are supplied by the institution.
- The data used for the demonstration is generated and verified. The declarations of availability supplied with the instance carry the value SYNTHETIC in their source column.
- The application depends on the OR-Tools library for the solving.
- No collection of validated timetables exists, so the weights of the criteria are initialised at the values held in the catalogue.
- The comparisons which serve to adjust the weights are produced by the use of the application itself.
- The examinations of a session concern the same population as the semester.

3. Functional requirements

3.1 Summary of the requirements

The table below restates, unchanged, the functional requirements defined in the Cahier des Charges, section 4.2. It is repeated here because section 3.2 specifies, requirement by requirement, the input, the processing and the output of each one; the two documents must be read as agreeing on this text, and this table is that point of agreement.

Table 3. Summary of the functional requirements

Code	Requirement	Priority
FR-1	Load and manage the data of the department	Necessary
FR-2	Allow a teacher to declare availability on a weekly grid	Necessary
FR-3	Generate a timetable respecting the hard constraints H1 to H12	Necessary
FR-4	Improve the quality criteria within a time limit	Necessary
FR-5	Produce several candidates and give each a score out of 100	Necessary
FR-6	Order the candidates by score	Necessary
FR-7	Display the timetable by teacher, by group and by room	Necessary
FR-8	Report the rules in conflict when no timetable exists, by the verifications of the data and by the explanation of infeasibility returned by the solver	Necessary
FR-9	Configure the calendar : holidays, closed slots and shortened-day period	Necessary
FR-10	Print or export a timetable view	Expected
FR-11	Authenticate the users and restrict the access by role	Necessary
FR-12	Verify the data before solving and report the structural risks found	Necessary
FR-13	Produce the candidates under distinct weight profiles	Necessary
FR-14	Compare two candidates criterion by criterion on the same screen	Necessary
FR-15	State the contribution of each criterion to the difference between two candidates	Necessary
FR-16	Recommend one candidate and state the rule which produced the recommendation	Expected
FR-17	Signal a recommended candidate which another candidate improves on every criterion	Expected
FR-18	Display the occupancy of each classroom and of each laboratory	Expected
FR-19	Record every generation run with its data, its seed, its weights and its results	Necessary
FR-20	Generate the timetable of an examination session	Expected (increment 2)
FR-21	Adjust the weights of the criteria from the comparisons made	Optional (increment 2)
FR-22	Explain in ordinary language the quality of a candidate, from figures already computed	Necessary

Code	Requirement	Priority
FR-23	Regenerate a timetable from an accepted recommendation, preserving every hard constraint	Necessary
FR-24	Answer a question asked in ordinary language about a run or a candidate	Necessary
FR-25	Produce a readable report on a run, ready to be printed or circulated	Expected

3.2 Detailed specification

Table 4. FR-1. Data management

Input	Processing	Output
Files or forms validated by the server	Verification of the types and of the references, then recording	Entities recorded and report of the rejected lines

Table 5. FR-2. Collection of availability

Input	Processing	Output
Slots marked on the weekly grid	Recording of one line per teacher and per slot, with the origin of the declaration	Availability table updated

Table 6. FR-3. Generation of a valid timetable

Input	Processing	Output
Data of the department and availability	Construction of the CP-SAT model then solving with all the workers	One slot and one room assigned to each session

Table 7. FR-4. Improvement of the criteria

Input	Processing	Output
Weights of the criteria and time limit	Minimisation of the weighted penalty within the limit	Best solution found when the limit is reached

Table 8. FR-5. Scoring of the candidates

Input	Processing	Output
Valid timetables produced and bounds of normalisation	Calculation of the sub-scores then of the overall score	List of candidates with their scores

Table 9. FR-7. Display of the views

Input	Processing	Output
Published timetable and chosen filter	Selection of the sessions concerning the resource	Weekly grid of the resource

Table 10. FR-8. Naming of the rules in conflict

Input	Processing	Output
Data of the department, then the result of the optimisation run	Arithmetic verifications before solving; then, if the run concludes that no timetable exists, a second run under assumptions, with a single worker and without objective	List of the rules in conflict, each one named by its code in the catalogue

Table 11. FR-9. Configuration of the calendar

Input	Processing	Output
Holidays, closed half-days and shortened-day period	Closing of the corresponding slots and shifting of the hours displayed	Slot grid updated, without modification of the code

Table 12. FR-11. Authentication and rights

Input	Processing	Output
Identifiers and role	Verification of the token and of the role at each request	Access authorised or refused

Table 13. FR-12. Verification of the data before solving

Input	Processing	Output
Data of the department and availability declared	Application of the five verifications described in section 8.2, without any solving	Report of the structural risks, with the resource concerned and the quantity missing

Table 14. FR-13. Production of the candidates under distinct profiles

Input	Processing	Output
Weight profiles, seed and total time limit	Solving repeated, one profile at a time, within the budget allotted to each	One candidate per profile, duplicates removed

Table 15. FR-14. Comparison of two candidates

Input	Processing	Output
Two candidates of the same run	Reading of the sub-scores recorded for each	Table of the criteria with the two values and their difference

Table 16. FR-15. Decomposition of the difference

Input	Processing	Output
Two candidates, their sub-scores and the weights of the run	Calculation of the contribution of each criterion according to section 6.7	Ordered list of the contributions, whose sum equals the difference of the scores

Table 17. FR-16. Recommendation

Input	Processing	Output
Ordered candidates of a run	Selection of the first, then verification of dominance	Candidate recommended and statement of the rule applied

Table 18. FR-19. Record of the runs

Input	Processing	Output
Parameters of the run and results obtained	Recording of the seed, of the weights, of the bounds and of the version of the model	Run consultable and reproducible

Table 19. FR-20. Examination session

Input	Processing	Output
Examinations, students, rooms, period of the session and supervisors	Construction of the model of section 6.8 then solving	One slot and one or more rooms assigned to each examination

Table 20. FR-21. Adjustment of the weights

Input	Processing	Output
Comparisons recorded and weights in force	Fitting described in section 6.9, then evaluation against the weights in force	Adjusted weights, or the weights in force kept and the result reported as not established

Table 21. FR-22. Explanation of a candidate

Input	Processing	Output
Sub-scores and contributions of the candidate	Context assembled, call to the language service, then verification that every figure of the answer appears in the context	Paragraph in ordinary language, or the computed form if the check fails or the service is unavailable

Table 22. FR-23. Regeneration from a recommendation

Input	Processing	Output
Candidate, recommendation accepted by the person in charge	Translation of the recommendation into a solver input, section 6.10, then a new call to the engine of FR-3 / FR-4	New candidate, linked to the recommendation and to the original run

Table 23. FR-24. Question in ordinary language

Input	Processing	Output
Question of the user and identifier of the run	Selection of the figures needed to answer, call to the language service, verification of the figures returned	Answer in ordinary language, or a statement that the question cannot be answered from the run

Table 24. FR-25. Report on a run

Input	Processing	Output
Run, its candidates, their scores and the timetable published	Assembly of the figures of the run into a context, call to the language service, verification of the figures returned	Report ready to be printed, or the computed tables if the service is unavailable

3.3 Main flow

1. The administrator loads the data and configures the calendar of the semester.
2. The teachers fill in their availability on the weekly grid.
3. The person in charge asks for the verification of the data and reads the report of the structural risks.
4. The person in charge launches the run and fixes the time limit.
5. The server records the run and executes it in the background, one weight profile at a time.
6. Each candidate obtained is scored, ordered and made visible as soon as it is available.
7. If no candidate can be obtained, the diagnosis run is executed and the rules in conflict are displayed.
8. The person in charge reads the AI recommendations attached to a candidate and, on acceptance, triggers a regeneration.
9. The person in charge compares two candidates, reads the contribution of each criterion to the difference, and the choice made is recorded.
10. The chosen timetable is published and becomes visible to the teachers and the students.

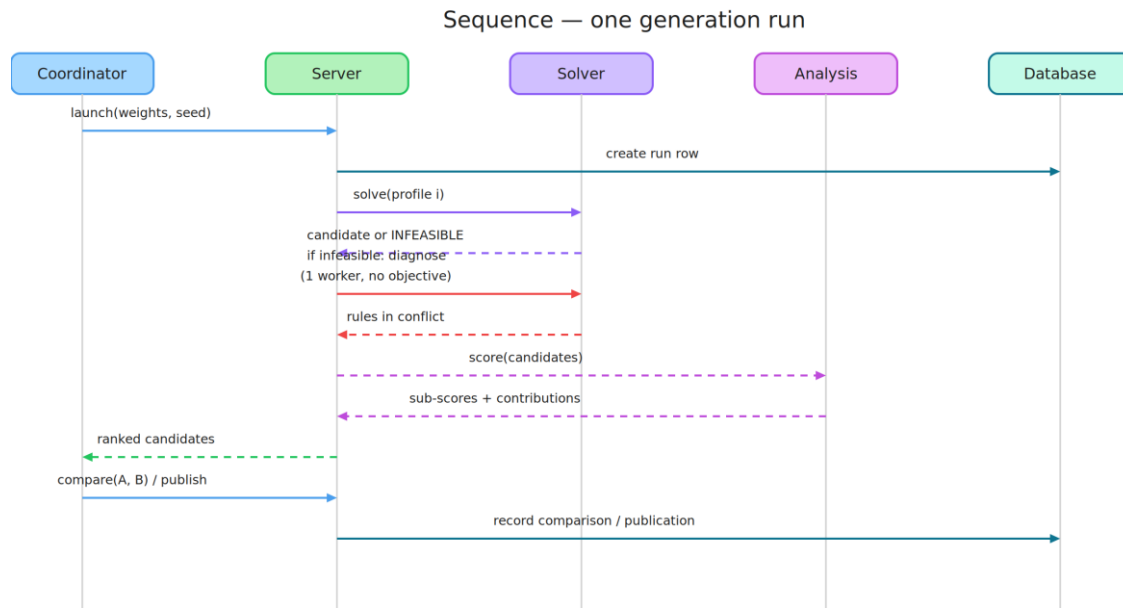


Figure 3. Sequence of one generation run, from launch to publication.

4. External interfaces

4.1 User interfaces

- **Availability grid** : weekly grid of days and periods, each slot marked available, unavailable or preferred, then saved.
- **Generation screen** : report of the verifications preceding the solving, weight profiles, time limit, launch control, state of the run and list of the candidates with their scores as they are produced.
- **Comparison screen** : two candidates side by side, table of the criteria, contribution of each criterion to the difference of the scores, indication of dominance where it applies, list of the AI recommendations attached to the leading candidate, and controls to retain a candidate or to accept a recommendation and trigger a regeneration.
- **Report of conflicts** : list of the rules named by the verifications of the data and by the diagnosis run, each one with its code and its statement.
- **Timetable view** : weekly grid filtered by teacher, group, classroom or laboratory, with printing of the displayed view.
- **Examination view** : calendar of the session by group and by room.
- **Assistant panel** : free-text question on the current run, answer returned in ordinary language, and controls to request an explanation of a candidate or a report on the run.
- **History of the runs** : list of the runs with their parameters, their candidates and the timetable published from them.
- **Administration screen** : management of the accounts, of the holidays, of the closed half-days and of the shortened-day period.

4.2 Software interfaces

- **Database** : access to PostgreSQL through an object-relational mapper, with prepared queries.
- **Solving engine** : call to the OR-Tools CP-SAT library in the same process as the server, in a background task.
- **Analysis components** : internal modules receiving the candidates as data and returning the scores and the explanations, without access to the model of the solver.
- **Fitting of the weights** : scikit-learn, used for the logistic regression of section 6.9.
- **Language model API** : external service reached over HTTPS. It receives a context of figures and a question, and returns text. It holds no credential for the database, is reached through a single adapter, and can be switched off by configuration.
- **Browser** : service of the interface and exchange of data in JSON format.

4.3 Communication interfaces

- Exchanges between browser and server in HTTPS.
- REST interface, data in JSON format.
- Authentication by token transmitted in the header of each request.
- The launch of a run answers immediately with the identifier of the run, the state of which is then asked for at regular intervals, no request being held open for the duration of the solving.

5. Data model

5.1 Main entities

The entities below correspond to the thirteen files of the reference instance, extended by the entities which the application creates during its own use.

Table 25. Main entities of the data model

Entity	Principal attributes	Role
Programme	code, label, degree cycle, department	Field of study of the LMD structure
Promotion	programme, level, academic year, number of students	Annual cohort of a programme
Group	promotion, parent group, type, label, number of students	Promotion, tutorial group or laboratory subgroup
Student	promotion, tutorial group, laboratory subgroup	Individual, used by the examination model
Teacher	department, rank, maximum hours per week	Person in charge of the sessions
Course	code, department, programme, level, semester, credits	Teaching unit of a programme
Session	course, group, teacher, type, duration, required room type, locked	Unit to be placed in the timetable

Entity	Principal attributes	Role
Room	building, code, capacity, type, equipment	Place where a session is held
Slot	day index, period index, start and end hours, open	Time unit of the weekly grid
Availability	teacher, slot, state, semester, source	Declaration made by a teacher or generated
Holiday	date, label, lunar, approximate, blocking	Day on which no session is placed
Calendar configuration	key, value	Periods per day, closed half-days, shortened-day window
Constraint	code, name, kind, default weight, XHSTT reference	Catalogue of the rules supported
Run	date, seed, time limit, state, version of the model	One execution of the generation
Weight profile	name, weight of each criterion	Set of weights used to produce a candidate
Candidate	run, profile, cost, score	Valid timetable produced by a run
Placement	candidate, session, slot, room	Assignment of one session inside one candidate
Sub-score	candidate, criterion, raw value, normalised value	Value of one criterion for one candidate
Comparison	run, candidate retained, candidate set aside, user, date	Choice recorded on the comparison screen
Recommendation	candidate, criterion, action type, parameters, status, resulting candidate	AI proposal, and its outcome once acted upon
Publication	candidate, date, user	Timetable made visible to the teachers and the students

Data model — main entities

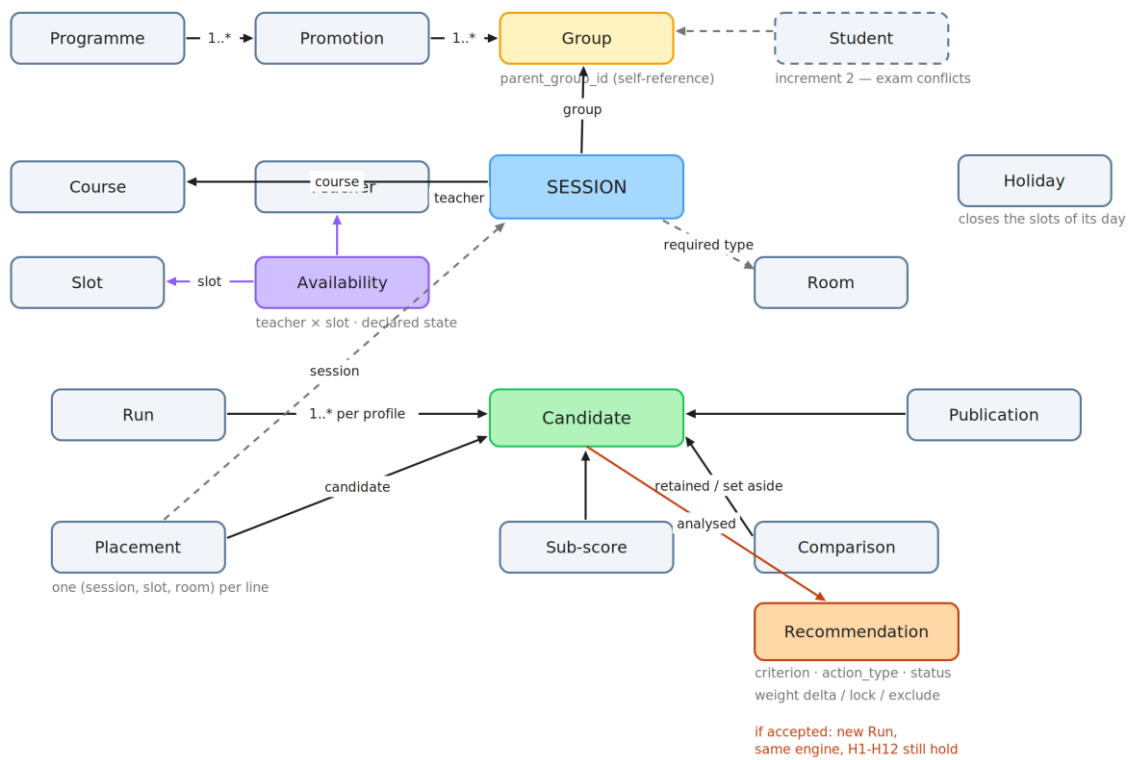


Figure 4. Entity-relationship diagram of the database.

5.2 Rules of the model

- **Central entity** : the session is the unit which the engine places, all the other entities describe it or constrain it.
- **Hierarchy of the groups** : a group refers to its parent group, which makes it possible to express the relation between a promotion and its subgroups. The chain admitted is promotion, then tutorial group, then laboratory subgroup.
- **Indexing of the slots** : each slot carries a single index calculated from the day and the period, which reduces the placement of a session to one integer variable.

$$\text{slot} = \text{day_index} \times \text{periods_per_day} + \text{period_index}$$

- **Configuration as data** : the holidays, the closed half-days and the shortened-day window are recorded in the configuration and not written in the code. They act by setting the field open of the slots concerned to false and by shifting the hours displayed, so that the model itself is untouched.
- **Origin of a declaration** : every line of availability carries the origin of the declaration, so that a value produced by the generator and a value entered by a teacher can always be told apart.
- **Immutability of a candidate** : a candidate is not modified after being recorded. A modification would leave the sub-scores describing a content which no longer exists.
- **Bounds recorded with the run** : the bounds used to bring the criteria back to the interval from zero to one are computed from the data of the run and recorded with it, so that a score can be recomputed later exactly as it was displayed.

- **Traceability of a publication** : a published timetable refers to the candidate, the run, the seed and the weights which produced it.

6. Constraint model

6.1 Decision variables

Table 26. Decision variables of the model

Variable	Type	Domain	Role
start[s]	Integer	Open slots compatible with s	Slot where the session s begins
room[s]	Integer	Rooms of the required type and sufficient capacity	Room assigned to the session s
iv[s]	Interval	Built on start[s] and the duration	Occupation of the session s in time
y[s][t]	Boolean	0 or 1	Indicates that the session s occupies the slot t

The interval variables are built from the start variable and the duration of the session. They are required because the constraint of non-overlap of the OR-Tools library operates on intervals and not on start and end variables taken separately.

6.2 Hard constraints

Table 27. Hard constraints and their construction

Code	Statement	Construction used
H1	A teacher has at most one session per slot	Non-overlap of the intervals of each teacher
H2	A group has at most one session per slot	Non-overlap of the intervals of each group
H3	A room hosts at most one session per slot	Non-overlap of the intervals of each room
H4	Room of the type required by the session	Restriction of the domain of room[s]
H5	Capacity of the room covering the group	Restriction of the domain of room[s]
H6	Availability declared by the teacher respected	Withdrawal of the slots from the domain of start[s]
H7	Each session placed exactly once	Exactly one true value among the y[s][t]
H8	No session extending across two days	Withdrawal of the slots at the end of the day
H9	Closed slots and holidays not used	Withdrawal of the slots from the domain of start[s]
H10	A locked session keeps the slot given to it	Fixing of start[s] and of room[s]
H11	Aggregate demand within the capacity of the rooms	Cumulative constraint per type of room

Code	Statement	Construction used
H12	A promotion and its subgroups never busy together	Non-overlap of the intervals of the whole hierarchy

Note : the constraints H4, H5, H6, H8, H9 and H10 are applied by reducing the domain of the variables before the search begins. This costs nothing during the solving and reduces the space to be explored, which is preferable to expressing them as constraints verified afterwards.

6.3 Order of the constraints and the calendar

The rules of the institution do not appear in the table above and this is deliberate. A closed half-day and a shortened-day period are held as configuration; they act by setting the field open of the corresponding slots to false, after which the constraint H9 removes those slots from the domain of every session. The shift of the hours changes the start and end hours attached to a period and not the index of the slot, so no variable and no constraint of the model is affected.

On the configuration supplied with the instance, the two afternoon periods of Saturday are closed and the Friday afternoon is open, which leaves twenty-eight open slots out of thirty. A shortened-day window is declared from 18 February to 19 March 2026 with a shift of sixty minutes. Changing any of these values changes the timetable produced without any modification of the code.

6.4 Objective function

The engine minimises the weighted sum of the penalties associated with the soft constraints.

$$\text{minimise } \Sigma (\text{weight}_i \times \text{violations}_i)$$

Table 28. Criteria of quality and default weights

Code	Criterion	Default weight
S2	Idle time inside the day of a group	0.25
S3	Idle time inside the day of a teacher	0.15
S4	Number of distinct days of presence	0.10
S5	Preferred windows declared by a teacher	0.20
S6	Rooms markedly under-used or over-used	0.10
S7	Spread of the sessions of one course across the week	0.10
S10	Midday break preserved	0.00

These default values are read from the catalogue and their sum is 0.90. A weight profile normalises them so that their sum equals one before any score is computed, which makes the scores of two runs comparable. A criterion of weight zero is measured and displayed but does not influence the order.

6.5 Size of the model

Table 29. Size of the model on the reference instance

Element	Value on the reference instance
Sessions to be placed	218

Element	Value on the reference instance
Open slots	28
Integer variables	436, that is start[s] and room[s] for each session
Interval variables	218, one per session
Boolean variables $y[s][t]$	at most 6 104, that is 218×28 , reduced by the pruning of the domains
Families of hard constraints	12, from H1 to H12
Criteria of quality	7, of which 6 with a non-zero default weight
Objective functions	1 per weight profile
Solvings per run	3, one per profile, executed one after the other
Candidates retained per run	at most 3, duplicates removed

The upper bound of 6 104 Boolean variables is the product of the number of sessions by the number of open slots. The effective number is lower, because the constraints H4 to H10 remove from the domain of each session the slots and the rooms which it cannot use before the search begins. The figure is recorded as an upper bound rather than as an estimate.

6.6 Weight profiles and portfolio

A run produces one candidate per weight profile. Three profiles are defined by default : a balanced profile taken from the catalogue, a profile which favours the students by raising the weight of S2, and a profile which favours the teachers by raising the weights of S3 and S5. The total time limit of the run is divided between the profiles, and a candidate identical to one already obtained is not retained.

The diversity of the portfolio is therefore obtained by varying the objective and not only the random seed. Two candidates produced under two profiles differ for a reason which is exactly the difference between those profiles, which is what makes the comparison of section 6.7 meaningful.

6.7 Score, order and decomposition of the difference

Each criterion i is measured on a candidate k by a raw value $v_i(k)$, which is the number of violations or the quantity penalised. This value is brought back to the interval from zero to one against bounds computed from the data of the run and recorded with it, the value one designating the best situation.

$$n_i(k) = 1 - (v_i(k) - \min_i) / (\max_i - \min_i)$$

The score of a candidate is the weighted sum of these normalised values, expressed out of 100.

$$\text{score}(k) = 100 \times \sum (w_i \times n_i(k))$$

The candidates are ordered by decreasing score. Equal scores are separated by the criteria taken in the order of their weights.

Because the score is a linear function of the normalised criteria, the difference between two scores decomposes exactly into a sum of terms, one per criterion.

$$\text{score}(A) - \text{score}(B) = 100 \times \sum (w_i \times (n_i(A) - n_i(B)))$$

The term relative to the criterion i is the contribution of that criterion to the difference. The list of these contributions is what the application displays as the explanation required by FR-15. This explanation is exact and not approximate : it is the calculation of the score itself, read term by term, and no separate explanatory method is needed. The identity between the sum of the contributions and the difference of the scores is verified by an automatic test, described in section 8.4.

The recommendation of FR-16 designates the candidate of highest score. A candidate which another candidate improves on every criterion is signalled, because that situation shows that the weights conceal a compromise instead of expressing one, and the person in charge must then decide with full knowledge of it.

6.8 Model of the examination session

The examination session reuses the same construction with different variables, an examination replacing a session. This part belongs to the second increment.

Table 30. Constraints of the examination model

Code	Statement	Construction used
X1	A student never has two examinations in the same slot	Non-overlap of the intervals of each student
X2	Capacity of the rooms covering the number of candidates	Sum of the capacities assigned to the examination
X3	Examination placed inside the period and outside the holidays	Restriction of the domain of the start variable
X4	No supervisor assigned to two examinations in the same slot	Non-overlap of the intervals of each supervisor
SX1	Examinations of the same group spread over the session	Penalty on the proximity of two examinations of the same group

Three differences with the weekly model must be noted. An examination may occupy several rooms at once, so the assignment of the rooms becomes a sum of capacities and not the choice of a single room. The horizon is the period of the session and not the week, so the index of the slot is computed over the days of that period. And the conflicts are computed at the level of the individual student and not of the group : for the weekly timetable a promotion follows a fixed programme and reasoning on groups loses no information, whereas two students of the same group may present different optional courses. The file of the 425 students of the instance is used for this purpose and for this purpose only.

6.9 Adjustment of the weights

Each comparison recorded provides two candidates and the one which was retained. The vector of the differences of their normalised criteria is formed, and a logistic regression without constant term is fitted on these vectors, the probability of retaining a candidate increasing with the difference of the scores.

$$P(\text{A retained rather than B}) = \text{logistic}(\sum (w_i \times (n_i(A) - n_i(B))))$$

Three restrictions are imposed on the fitting :

- the weights remain non-negative, so that reducing the violations of a criterion never lowers the score;
- the weights are renormalised to a sum of one after the fitting, which fixes the scale of the score without modifying the order of any candidate, a positive multiple of the weights giving the same order;
- the fitting is regularised towards the weights in force, which remain the reference as long as the number of comparisons is small.

The adjusted weights replace the weights in force only if the evaluation of section 8.5 concludes in their favour. Otherwise the weights in force are kept and the result is recorded as not established.

6.10 Recommendation and regeneration model

The AI analysis layer scores every candidate against the criteria of section 6.7. Where a criterion carries most of the gap between the leading candidate and a theoretical maximum, or where a dominance test of a nearby profile would improve on it, the layer proposes a recommendation. A recommendation is one row taken from a fixed catalogue of three action types.

Table 31. Catalogue of the recommendation action types

Action type	Parameters	Effect on the next run
weight_delta	criterion code, new weight	Replaces one weight of the profile used to obtain the candidate
lock_session	session id	Fixes start[s] and room[s] to their value in the candidate, as for H10
exclude_slot	session id, slot id or room id	Withdraws one value from the domain of start[s] or room[s]

A recommendation is stored with the status proposed. If the person in charge accepts it, the server builds a new run whose weight profile or whose domain restrictions are updated by exactly that one action, and calls the solving step of FR-3 and FR-4 again. The new candidate is recorded under a new run, linked to the recommendation and to the original run, and it is scored, ordered and displayed exactly as any other candidate. If the recommendation is refused, its status becomes rejected and no run is created.

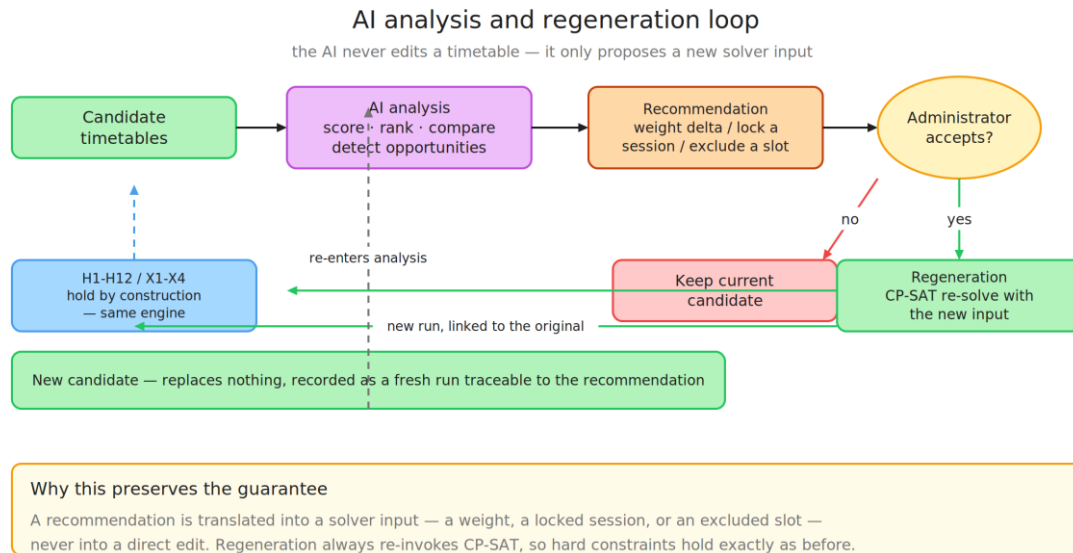


Figure 5. AI recommendation and regeneration loop.

Because a recommendation is restricted to these three action types, it can only ever change what the solver is asked to do, never what a solver call is allowed to return. The hard constraints H1 to H12, and X1 to X4 for the examination model, are declared identically in the regenerated run, so they hold in the new candidate for the same reason they held in the candidate which produced the recommendation.

6.11 AI assistant service

The assistant is an adapter around an external language model API. It is stateless between calls and holds no credential for the database.

6.11.1 Construction of the context

For each request the context builder assembles a JSON object containing only what is needed to answer, and nothing else :

Table 32. Context supplied to the language model

Request	Context supplied	Size
Explain a candidate	sub-scores, normalised values, weights in force, overall score	one candidate
Compare two candidates	the two sets of sub-scores and the contributions of section 6.7	two candidates
Answer a question	placements concerned by the question, scores of the run, calendar in force	bounded by the run
Produce a report	run parameters, all candidates with their scores, the timetable published	one run

6.11.2 Verification of the answer

The answer returned is parsed and every numeric token it contains is compared with the numbers present in the context. The rule applied is the following.

numbers(answer) $\subseteq$ numbers(context)

If the inclusion does not hold, the answer is rejected and the computed form is displayed instead. The same fallback applies when the service returns an error or does not answer within the time allowed.

This verification detects a figure the model invented. It does not establish that the rest of the sentence is correct, and no requirement of this document depends on it doing so. It is the reason the assistant is excluded from every calculation of section 6.7 and from every decision recorded in section 5.1.

6.11.3 Suggestions produced by the assistant

A suggestion phrased by the assistant is not executable in that form. Before being offered to the person in charge it is matched against the catalogue of section 6.10; a suggestion which matches no action type of that catalogue is displayed as a remark and carries no control to act upon it.

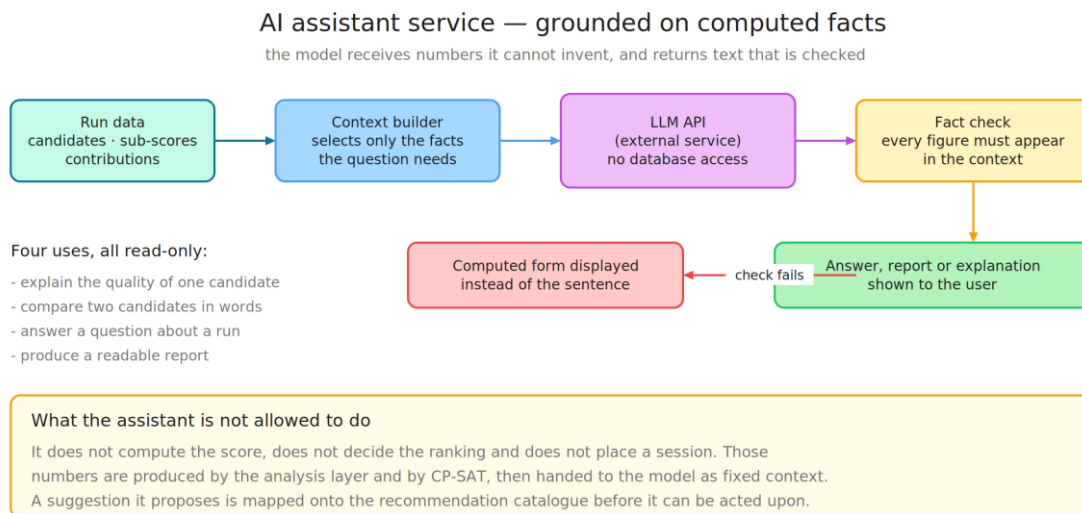


Figure 6. AI assistant service, grounded on computed facts

7. Non-functional requirements

7.1 Performance

- Target of a first valid timetable in less than 60 seconds on the reference instance.
- Target of a portfolio of three candidates within a configurable limit of 5 minutes.
- Decomposition of the difference between two candidates in less than 1 second.
- Response of the interface in less than 500 milliseconds outside the generation and outside a call to the assistant.
- Answer of the assistant within a configurable timeout, defaulting to 10 seconds as stated in the Cahier des Charges, beyond which the computed form is displayed.
- Capacity of one faculty, that is about 220 sessions, 45 teachers and 20 rooms.

The two first figures are objectives to be measured during the second phase and not guarantees. The problem belongs to a class for which no bound on the solving time can be announced in advance, which is why the engine works under a time limit and returns the best result obtained when the

limit is reached. The diagnosis run is excluded from these targets, since it uses a single worker by obligation.

7.2 Reliability

- A timetable is returned whenever one exists.
- An instance without solution produces a report of the rules in conflict and never an indefinite search.
- For the same data, the same weights and the same random seed, two executions give the same result.
- The unavailability of the optional language service removes no function other than the formulation of the sentence.

7.3 Security

- Authentication by token, passwords recorded in hashed form.
- Verification of the rights on the server side at each request.
- Access to the database through prepared queries.
- Exchanges encrypted between the browser and the server.
- A teacher is identified by an account, a student is represented by the group to which the student belongs outside the examination module, and no information foreign to the construction of the timetable is recorded.

7.4 Maintainability and portability

- Calendar, periods, holidays and catalogue of the constraints recorded as data.
- Schema of the database versioned by migrations.
- Execution on any standard processor, without specific hardware.

7.5 Requirements applicable to the analysis and learning components

These requirements apply to the components which score, order, explain and adjust the weights. They are stated separately because they concern properties which the other layers do not have to satisfy.

- **Separation** : no component of this layer may add, move or remove a session, nor modify a candidate already recorded.
- **Exactness** : every figure displayed is recomputable from the sub-scores, the bounds and the weights recorded with the run.
- **Determinism** : the same inputs give the same order and the same contributions, whatever the order in which the candidates are read.
- **Monotonicity** : reducing the violations of one criterion, all else being equal, never lowers the score.
- **Human control** : an adjustment of the weights is applied only after an explicit validation, and the weights in force are displayed with the run which used them.

- **Guarantee under regeneration** : the recommendation module writes only into the three fields listed in section 6.10; it never writes a placement. A regenerated candidate is therefore the output of an ordinary call to FR-3 and FR-4, and it is verified by the same tests as any other candidate.
- **Grounding of the assistant** : the language model receives only the context of section 6.11.1 and returns only text. Every figure it returns is verified against that context before display.
- **Independence from the assistant** : generation, scoring, ranking, comparison, regeneration and publication are all available when the language service is switched off or unreachable.
- **Confidentiality of the context** : the context sent to the external service contains identifiers, scores and placements. It carries no student name and no personal datum beyond the account of a teacher already responsible for a session.
- **Absence of free text in the decision** : the language service receives a structured explanation and returns a sentence, and the content of that sentence enters into no calculation.

8. Validation and tests

8.1 Validation of the engine

- Solving of the instances of the curriculum-based track of the competition of 2007, for which published results exist.
- Verification that the timetables produced contain no violation of a hard constraint.
- Comparison of the cost obtained with the best known results.
- For the examination session, the same procedure applied to the instances of the examination track, after their verification.

8.2 Verification of the data

Five verifications are applied to the data before any solving, in order to distinguish an instance without solution from an error in the model. The results obtained on the reference instance are given below.

Table 33. Result of the five verifications of the data

Verification	Result on the reference instance
Each session finds a room of the required type and of sufficient capacity	0 session without a suitable room
The open slots cover the demand for each type of room	lecture theatres 57%, classrooms 29%, computer laboratories 95%, science laboratories 86%
No teacher exceeds the maximum load of the rank	0 teacher above the limit, heaviest load 12 periods, that is 18 hours
Each teacher keeps enough free slots for the sessions assigned	0 teacher in difficulty, smallest margin 11 free slots
The hierarchy of the groups is consistent	0 invalid reference, 0 invalid chain, 425 students matching the declared group sizes

Each verification which fails names the resource concerned and the quantity which is missing, which is the content of the report required by FR-12. The occupancy of the computer laboratories at 95 per cent is the tightest point of the instance and the figure to watch when the data is modified.

8.3 Diagnosis of an infeasible instance

When the verifications pass and the optimisation run nevertheless concludes that no timetable exists, a second run is executed in order to name the rules responsible. The constraints are declared under enforcement literals which are given to the solver as assumptions, and the solver returns a subset of them sufficient to explain the infeasibility.

This run obeys three rules imposed by the solver and by the nature of the mechanism.

- **A single worker** : solving under assumptions does not admit parallelism, so the diagnosis is slower than a normal solving and is never used as the ordinary path.
- **No objective** : in the presence of a function to minimise the whole set of assumptions is returned, which removes any usefulness from the mechanism. The diagnosis run therefore drops the objective.
- **A sufficient set, not a minimal one** : the subset returned is reduced by heuristics and is not guaranteed to be the smallest. The interface presents it as a set of rules sufficient to explain the conflict. Obtaining a minimal set would require minimising the sum of the literals instead, which is recorded as a possible improvement.

8.4 Validation of the ranking and of the explanation

The analysis layer is verified by properties tested on candidates generated at random, a property holding for every input while an example holds for one.

Table 34. Properties tested on the analysis layer

Property	Statement tested
Exactness of the decomposition	The sum of the contributions equals the difference of the two scores
Invariance of the order	The order does not depend on the order in which the candidates are read
Monotonicity	Reducing the violations of one criterion never lowers the score
Detection of dominance	A candidate improved on every criterion is signalled
Fidelity of the formulation	Every figure of the sentence produced appears in the structured explanation

8.5 Evaluation of the adjustment of the weights

The comparisons collected are evaluated by leaving one out at a time and predicting it with the weights fitted on the others. The accuracy so obtained is compared with that of the weights in force, applied to the same comparisons.

The adjusted weights are adopted only if their accuracy exceeds that of the weights in force and if the number of comparisons reaches the threshold fixed before the experiment. Otherwise the

weights in force are kept and the result is reported as not established. This criterion is stated in advance and can be failed, which is the condition for the conclusion to have a meaning.

8.6 Acceptance tests

Table 35. Acceptance tests

Requirement	Test	Expected result
FR-2	Filling in of the grid	Availability recorded in less than 5 minutes
FR-3	Generation on the reference instance	No violation of a hard constraint
FR-5	Reading of a candidate	Overall score and sub-scores displayed
FR-8	Generation on an instance made infeasible on purpose	Report naming the rules in conflict by their code
FR-9	Closing of a half-day in the configuration	The corresponding slots disappear from every timetable produced, without modification of the code
FR-11	Connection with a teacher account	Access limited to own data
FR-12	Verification on an instance whose rooms are insufficient	The resource concerned and the quantity missing are named
FR-13	Run on the reference instance	Three distinct candidates produced
FR-15	Comparison of two candidates	The sum of the contributions equals the difference of the scores
FR-19	Repetition of a run with the same seed and the same weights	Identical candidates in the same order
FR-23	Acceptance of a weight_delta recommendation on the reference instance	New run created, new candidate satisfies H1 to H12, linked to the recommendation
FR-24	Question on a run whose answer is not in the context	The assistant states that it cannot answer, and invents no figure
FR-22, FR-25	Language service switched off in the configuration	Explanations and reports fall back to their computed form, every other function unaffected

9. Traceability

Table 36. Traceability of the requirements

Requirement	Cahier des Charges	This document	Element which implements it
FR-1	Section 4.2	3.2 and 5.1	Entities of the data model
FR-2	Section 4.2	3.2 and 5.1	Availability entity
FR-3	Section 4.2 and 5.1	6.1 and 6.2	Variables and hard constraints H1 to H12
FR-4	Section 4.2	6.4	Objective function
FR-5	Section 4.2	6.7	Calculation of the score
FR-6	Section 4.2	6.7	Order by decreasing score

Requirement	Cahier des Charges	This document	Element which implements it
FR-7	Section 4.2	4.1 and 5.1	Placement entity and views
FR-8	Section 4.2	3.2 and 8.3	Verifications of the data and diagnosis run
FR-9	Section 4.2 and 5.2	5.2 and 6.3	Calendar held as configuration
FR-11	Section 4.2	4.3 and 7.3	Token and verification of the rights
FR-12	Section 4.2	8.2	Five verifications preceding the solving
FR-13	Section 4.2 and 5.4	6.6	Weight profiles and portfolio
FR-14	Section 4.2	4.1	Comparison screen
FR-15	Section 4.2	6.7	Decomposition of the difference
FR-16	Section 4.2	6.7	Rule of recommendation
FR-17	Section 4.2	6.7 and 8.4	Test of dominance
FR-18	Section 4.2	4.1 and 5.1	Views by classroom and by laboratory
FR-19	Section 4.2	5.1 and 5.2	Run entity and traceability of a publication
FR-20	Section 4.2 and 5.5	6.8	Model of the examination session
FR-21	Section 4.2	6.9 and 8.5	Fitting of the weights and its evaluation
FR-22	Section 4.2 and 7.5	3.2 and 6.11	Assistant service, explanation of a candidate
FR-23	Section 4.2, 4.4 and 6	3.2 and 6.10	Recommendation catalogue and regeneration call
FR-24	Section 4.2 and 7.5	3.2 and 6.11	Context builder and verification of the figures
FR-25	Section 4.2 and 7.5	3.2 and 6.11	Report assembled from the figures of the run

10. Annexes

- Entity-relationship diagram of the database, including the Recommendation entity.
- Diagram of the division of the responsibilities between the layers.
- Sequence diagram of one generation run.
- Diagram of the AI recommendation and regeneration loop.
- Diagram of the AI assistant service and of its grounding.
- Diagram of the actors and of the main use cases.
- Catalogue of the constraints referenced to the XHSTT types.

- Generator of the reference instance and the thirteen files produced.
- Report of the five verifications of the instance.
- Results of the validation on the published instances.
- Editable sources of the figures, in Excalidraw format.